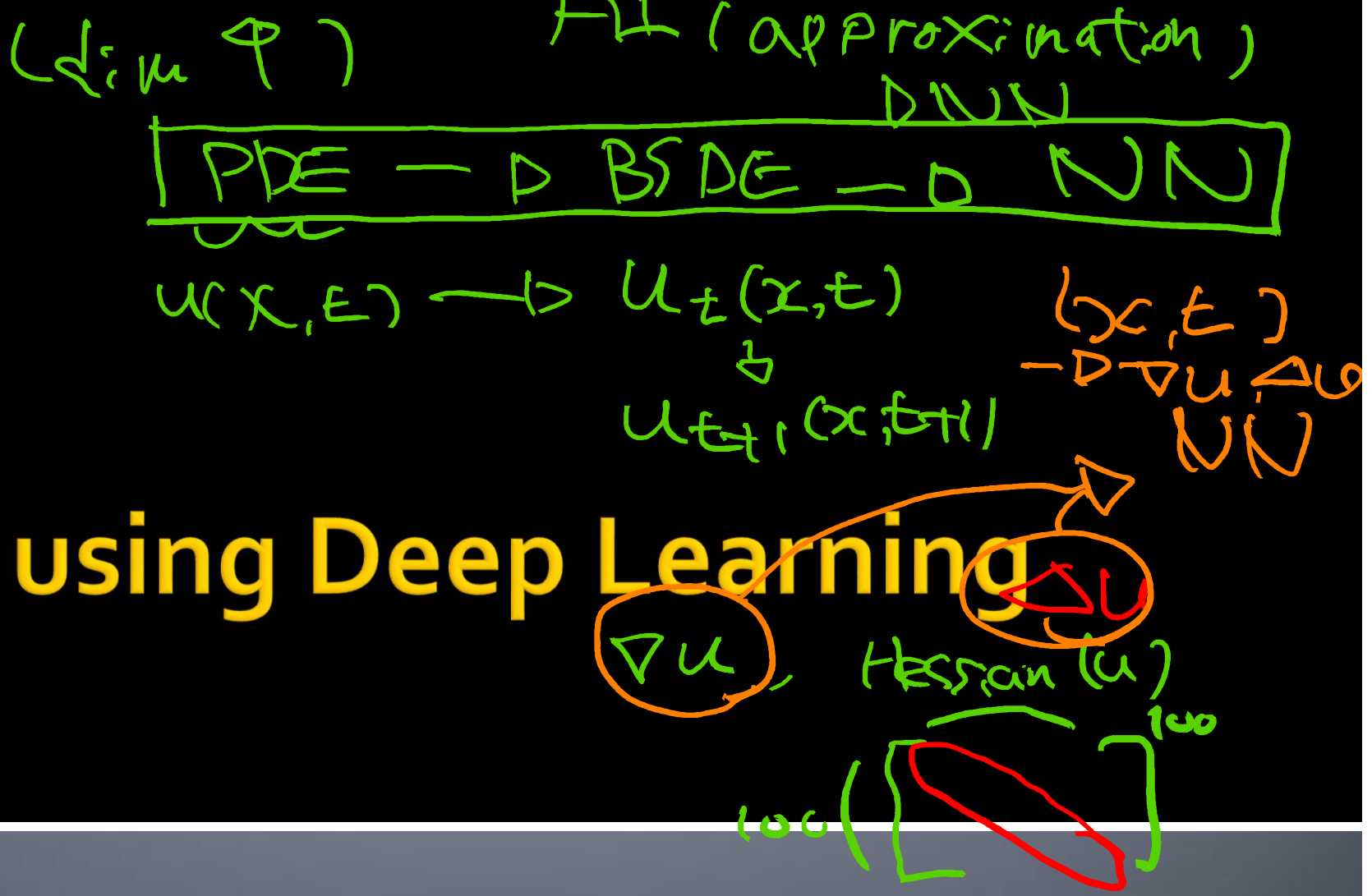
56262

PDE using Deep Learning



Solving High-Dimensional Partial Differential Equations Using Deep Learning (*Proceedings of the National Academy of Sciences, 2018*)

tr (Hessian) = Laplacian

Problem Statement 1/3

- **Semilinear parabolic PDE**

$$\frac{\partial u}{\partial t}(t,x) + \frac{1}{2}Tr(\sigma\sigma^T Hess_x u) + \nabla u(t,x)\cdot\mu(t,x) + f(t,x,u(t,x),\sigma^T\nabla u(t,x)) = 0 \quad (1)$$

- **Notations**
  - $u(t,x)$: unknown solution
  - $x \in \mathbb{R}^d$
  - d can be very large (e.g. 100 dimensions)
  - $u(T,x) = g(x)$: terminal condition (2)

$0 \leq t \leq T$

Problem Statement 2/3

- **Semilinear parabolic PDE**

$$\frac{\partial u}{\partial t}(t,x) + \frac{1}{2}Tr(\sigma\sigma^T Hess_x u) + \nabla u(t,x)\cdot\mu(t,x) + f(t,x,u(t,x),\sigma^T\nabla u(t,x)) = 0$$

- **Applications**

  - Black-Scholes equations → Financial derivative
  - Hamilton-Jacobi-Bellman equations → Optimal control
  - Allen-Cahn equations. → Phase transition

Problem Statement 3/3

- As elegant as their PDE models are, their practical use has proven to be very limited due to the curse of dimensionality.
- The computational cost for solving them goes up exponentially with the dimensionality.

$$\frac{\partial u}{\partial t}(t,x) + \frac{1}{2}Tr(\sigma\sigma^T Hess_x u) + \nabla u \cdot \mu + f(t,x,u,\sigma^T \nabla u) = 0$$

- $u(t,x)$: unknown solution
- $x \in \mathbb{R}^d$
- d can be very large (e.g., 100 dimensions)

Basic Idea

PDE → BSDE → Neural Network training

- This paper introduces a deep learning-based approach that can handle general high-dimensional parabolic PDEs.

Partial Differential Equations (PDE)

↓

Backward Stochastic Differential Equations (BSDE)

↓

The gradient of the unknown solution is approximated by NN

BSDE 1/3

- Using the nonlinear Feynman–Kac representation, the PDE solution satisfes a **Backward SDE**:

Partial Differential Equations (PDE)

$$\frac{\partial u}{\partial t}(t,x) + \frac{1}{2} Tr(\sigma\sigma^T Hess_x u) + \nabla u \cdot \mu + f(t,x,u,\sigma^T \nabla u) = 0$$

Backward Stochastic Differential Equations (BSDE)

$$u(t, X_t) = u(0, X_0) - \int_0^t f(s, X_s, u, \sigma^T \nabla u) ds + \int_0^t (\nabla u)^T \sigma dW_s$$

The BSDE reformulation turns the PDE into a **recursive time-stepping equation**, so that u can be **updated forward along simulated stochastic paths X_t**.

BSDE 2/3

- Backward Stochastic Differential Equations (BSDE)

$$u(t, X_t) = u(0, X_0) - \int_0^t f(s, X_s, u, \sigma^T \nabla u) ds + \int_0^t (\nabla u)^T \sigma dW_s$$

- **Simulated Stochastic Paths X_t** $0 = t_0 < t_1 < \ldots < t_N = T$

$$X_{t_{n+1}} = X_{t_n} + \mu(t_n, X_{t_n})\Delta t_n + \sigma(t_n, X_{t_n})\Delta W_n$$

Drift term (deterministic)

Diffusion term (stochastic)

Brownian motion

- **Discretized BSDE**

$$u(t_{n+1}, X_{t_{n+1}}) \approx u(t_n, X_{t_n}) - f(\cdot)\Delta t + (\nabla u)^T \sigma \Delta W_n$$

BSDE 3/3

- Discretized BSDE

$$u(t_{n+1}, X_{t_{n+1}}) \approx u(t_n, X_{t_n}) + f(\cdot)\Delta t + (\nabla u)^T \sigma \Delta W_n$$

This provides a **forward iterative scheme** for computing u.

- **Neural Network Approximation**
  - The unknown quantity is $\sigma^T \nabla u(t, x)$
  - The paper approximates it using a **neural network**:

$$\sigma^T \nabla u(t_n, X_{t_n}) \approx (\sigma^T \nabla u)(t_n, X_{t_n} | \theta_n)$$

  - Each time step t_n has its own **sub-network**.

Deep BSDE Sub-networks 1/2

- The structure works as follows:
  1. Simulate a path $X_{t_0}, \ldots, X_{t_N}$

$$X_{t_{n+1}} - X_{t_n} \approx \mu(t_n, X_{t_n})\,\Delta t_n + \sigma(t_n, X_{t_n})\,\Delta W_n,$$

  2. Use a neural network to estimate gradients

$$\sigma^T \nabla u(t_n, X_{t_n}) \approx (\sigma^T \nabla u)(t_n, X_{t_n} | \theta_n)$$

  3. Propagate the solution forward through Discretized BSDE

$$u(t_{n+1}, X_{t_{n+1}}) \approx u(t_n, X_{t_n}) - f(\cdot)\Delta t + (\nabla u)^T \sigma \Delta W_n$$

Deep BSDE Sub-networks 2/2

- The structure works as follows:
  4. Produce an estimate of $u(T, X_T)$
  5. The PDE specifies the terminal condition $u(T, x) = g(x)$

  The loss function is

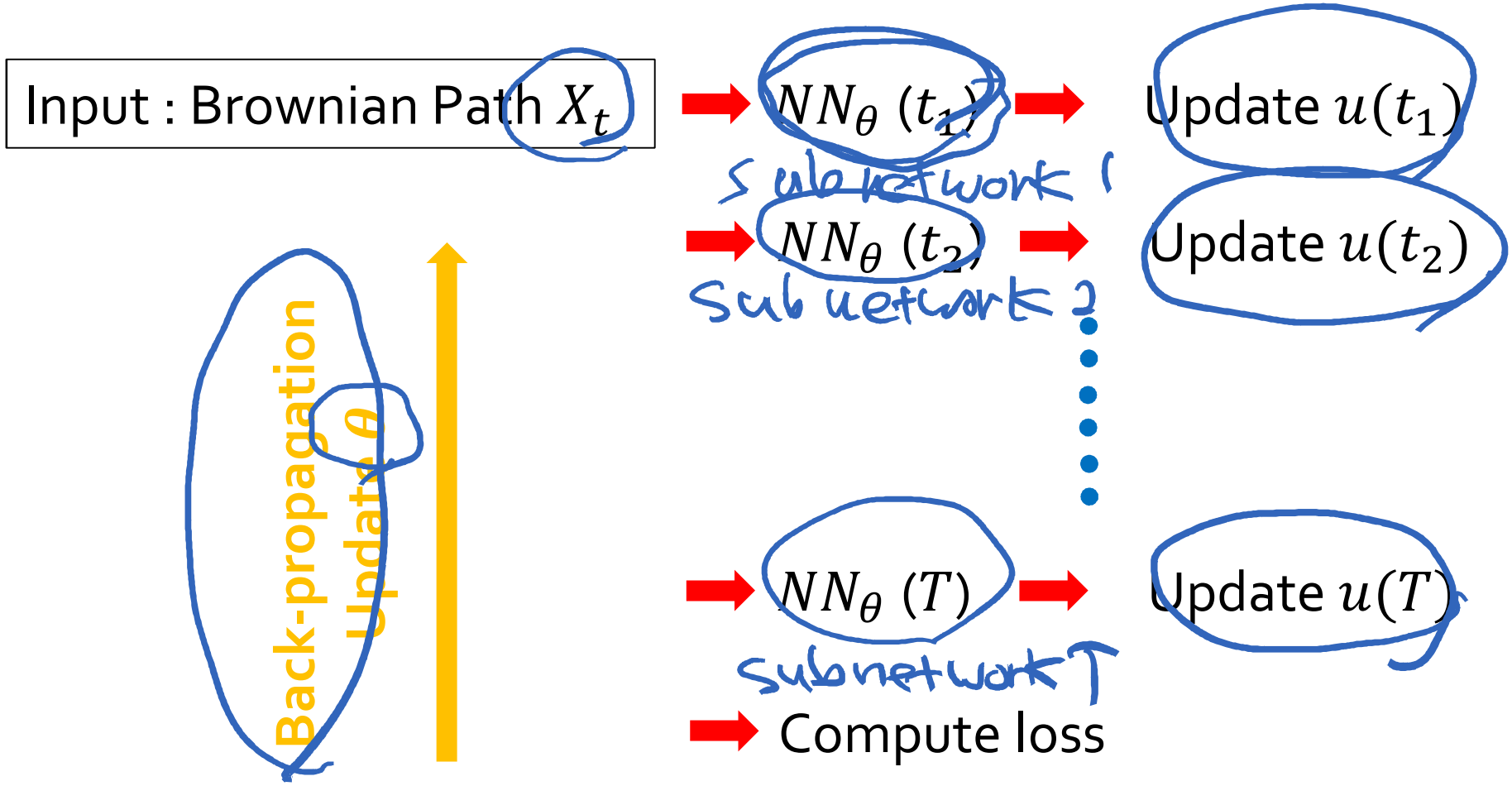
$$l(\theta) = E\left[|g(X_T) - \hat{u}(X_T)|^2\right]$$

PDE u approximates

$u(0, x_0) \to \cdots \to u(t, X_t) \to u(T, X_t)$

$l(\theta) = E\left[|g(X_T) - u(T, X_t)|^2\right]$

Pipeline



$$l(\theta) = E\left[|g(X_T) - \hat{u}(X_T)|^2\right]$$

$l(\theta)$ = | GT − estimated |^2

Final Intuition

- The methodology transforms PDE solving into a **machine learning problem**.
- Instead of computing the solution on a grid, the method
  - Simulates stochastic paths
  - Uses neural networks to approximate gradients
  - Enforces the terminal condition through a loss function.
- This approach avoids the **curse of dimensionality**, allowing PDEs with **very high dimensions (e.g., 100+)** to be solved efficiently

Black-Scholes Equation 1/3

- **Deep BSDE method** can solve **high-dimensional nonlinear PDEs in financial applications**

$$\frac{\partial u}{\partial t} + \bar{\mu} x \cdot \nabla u + \frac{\bar{\sigma}^2}{2} \sum_{i=1}^{d} x_i^2 \frac{\partial^2 u}{\partial x_i^2} - (1-\delta) Q(u) u - Ru = 0$$

nonlinear

- x_i: price of the i-th underlying asset 자산가격
- R: risk-free interest rate 이자율
- σ: volatility 변동성
- δ: recovery rate 회수율
- $Q(u)$: default intensity (decreasing function) 부도율

Black-Scholes Equation 2/3

- **Deep BSDE method** can solve **high-dimensional nonlinear PDEs in financial applications**

$$\frac{\partial u}{\partial t} + \bar{\mu} x \cdot \nabla u + \frac{\bar{\sigma}^2}{2} \sum_{i=1}^{d} x_i^2 \frac{\partial^2 u}{\partial x_i^2} - (1-\delta) Q(u) u - Ru = 0$$

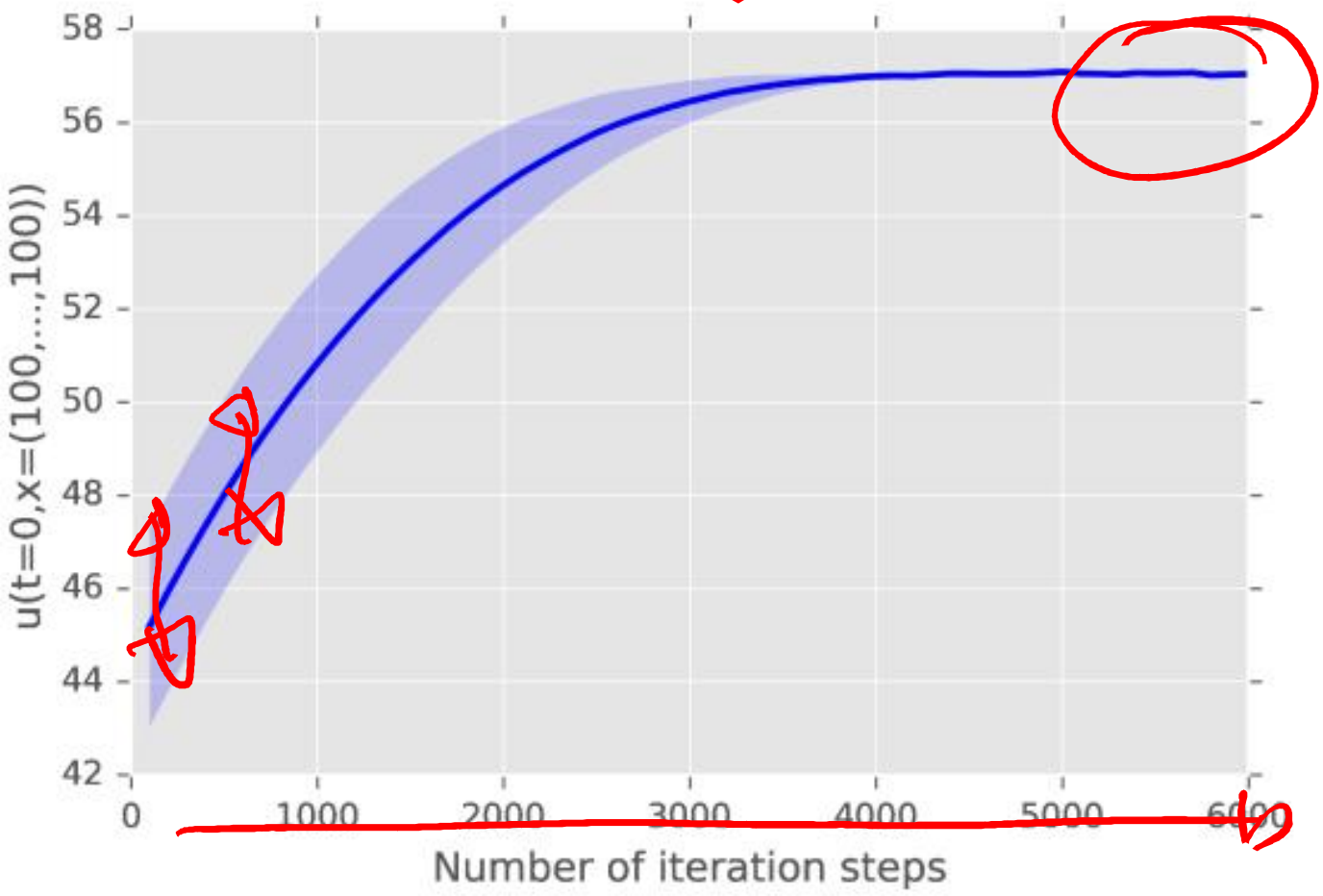
- number of assets $d = 100$
- volatility $\bar{\sigma} = 0.2$
- drift $\bar{\mu} = 0.02$
- interest rate $R = 0.02$.

- **Terminal Condition (payoff)** 만기시 받는 금액

$$u(T, x) = g(x) \qquad g(x) = \min(x_1, x_2, \ldots, x_{100})$$

Black-Scholes Equation 3/3

- The deep BSDE method achieves a relative error of size 0.46% in a runtime of 1607 seconds



Hamilton-Jacobi-Bellman Equation 1/5

- The **Hamilton–Jacobi–Bellman (HJB) equation** arises in **optimal control theory**. It describes how to determine the **optimal strategy that minimizes a cost over time**.

- **Optimal Control Problem**

$$dX_t = 2\sqrt{\lambda}\, m_t\, dt + \sqrt{2}\, dW_t$$

- X_t: state process
- m_t: control process
- W_t: Brownian motion
- λ: parameter representing control strength.

Hamilton-Jacobi-Bellman Equation 2/5

- The **Hamilton–Jacobi–Bellman (HJB) equation** arises in **optimal control theory**. It describes how to determine the **optimal strategy that minimizes a cost over time**.

- The objective is to **minimize a cost functional**

$$J(\{m_t\}) = E\left[\int_0^T \|m_t\|^2 dt + g(X_T)\right]$$

Cost

- **running cost** $\|m_t\|^2$ (cost of applying control)
- **terminal cost** $g(X_T)$ (cost depending on the final state).

Hamilton-Jacobi-Bellman Equation 3/5

- The **Hamilton–Jacobi–Bellman (HJB) equation** arises in **optimal control theory**. It describes how to determine the **optimal strategy that minimizes a cost over time**.

- **Value function** as the **minimum expected future cost when the state is x at time t**

$$u(t,x) = \min_{m_t} E\left[\int_0^T \|m_t\|^2 dt + g(X_T)\right]$$

Cost

Control

- time evolution of the cost
- stochastic dynamics of the system
- the effect of the optimal control.

Hamilton-Jacobi-Bellman Equation 4/5

- The **Hamilton–Jacobi–Bellman (HJB) equation** arises in **optimal control theory**. It describes how to determine the **optimal strategy that minimizes a cost over time**.

- Using **Itô formula(stochastic Taylor expansion)**, the value function satisfies the **Hamilton–Jacobi–Bellman PDE**

$$\frac{\partial u}{\partial t}(t,x) + \Delta u(t,x) - \lambda \|\nabla u(t,x)\|^2 = 0$$

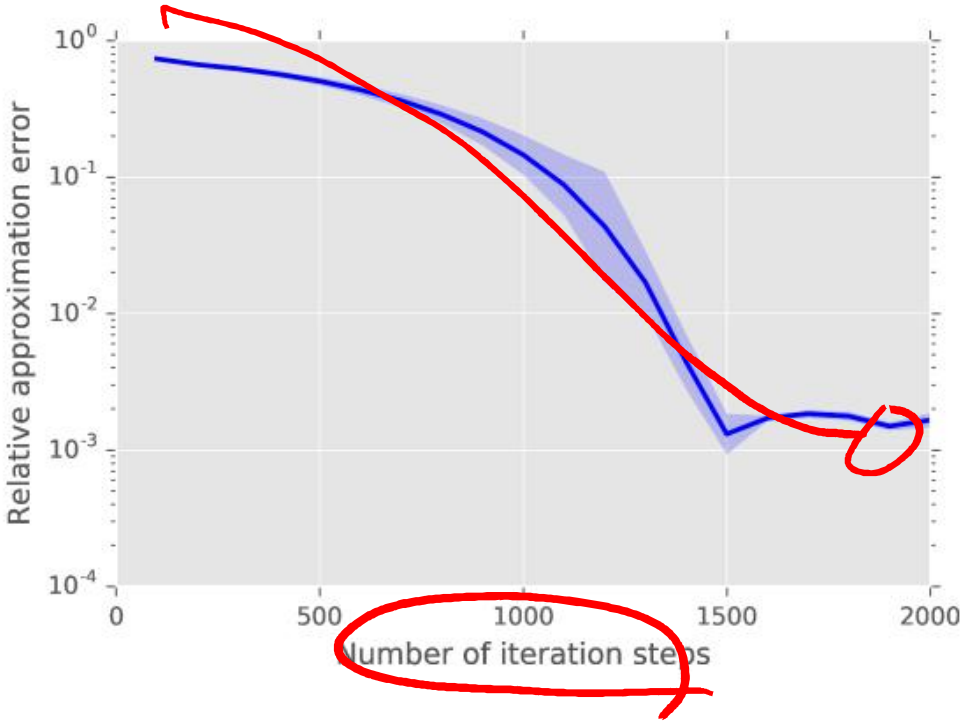
nonlinear

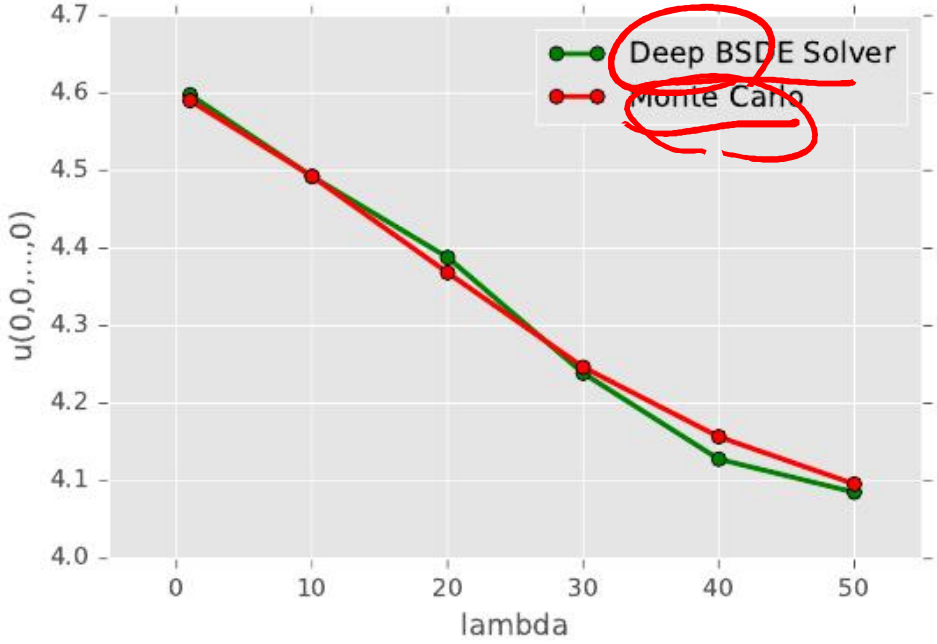
with terminal condition

$$u(T,x) = g(x)$$

Hamilton-Jacobi-Bellman Equation 5/5

- The deep BSDE method achieves a relative error of 0.17% in a runtime of 330 seconds





Allen-Cahn Equation 1/3

- The **Allen–Cahn equation** is a **reaction–diffusion partial differential equation** that arises in physics, particularly in the modeling of **phase separation and order–disorder transitions in materials**.

Diffusion term (spatial spreading or smoothing of the state)

$$\frac{\partial u}{\partial t}(t,x) = \Delta u(t,x) + u(t,x) - [u(t,x)]^3$$

Nonlinear (reaction term)

- $u(t,x)$ is the unknown function
- Δu is the Laplacian representing diffusion
- $u - u^3$ is a nonlinear reaction term.

Allen-Cahn Equation 2/3

- The **Allen–Cahn equation** is a **reaction–diffusion partial differential equation** that arises in physics, particularly in the modeling of **phase separation and order–disorder transitions in materials**.

$$\frac{\partial u}{\partial t}(t,x) = \Delta u(t,x) + u(t,x) - [u(t,x)]^3$$

with the ~~initial~~ final condition

$$u(T,x) = g(x) \qquad g(x) = \frac{1}{2 + 0.4\|x\|^2}$$

Allen-Cahn Equation 3/3

- The deep BSDE method achieves a relative error of 0.30% in a runtime of 647 seconds

